

How Do Semantically Equivalent Code Transformations Impact Membership Inference on LLMs for Code?

Anonymous Author(s)

Abstract

The success of large language models for code relies on vast amounts of code data, including public open-source repositories, such as GitHub, and private, confidential code from companies. This raises concerns about intellectual property compliance and the potential unauthorized use of license-restricted code. While membership inference (MI) techniques have been proposed to detect such unauthorized usage, their effectiveness can be undermined by semantically equivalent program transformation techniques, which modify code syntax while preserving functionality.

In this work, we systematically investigate whether semantically equivalent program transformation rules might be leveraged to evade MI detection. The results reveal that model accuracy drops by only 1.5% in the worst case, demonstrating that transformed datasets can effectively serve as substitutes for fine-tuning. Additionally, we find that one of the rules (RenameVariable) reduces MI success by 8.01%, highlighting its potential to obscure the presence of restricted code. To validate these findings, we conduct a causal analysis confirming that variable renaming has the strongest causal effect in disrupting MI detection, while other transformations have only moderate impacts. Notably, we find that combining multiple transformations does not further reduce MI effectiveness. Our results expose a critical loophole in license compliance enforcement for training large language models for code, showing that MI detection can be substantially weakened by transformation-based obfuscation techniques.

1 Introduction

Recent advancements in *large language models for code* (LLMs4Code) have significantly improved automated code completion, generation, and understanding [18, 20, 25]. These models employ complex architectures and are trained on extensive code corpora, often sourced from open-source repositories like GitHub [1, 7, 10]. However, their dependence on large-scale data raises critical concerns about intellectual property compliance and the potential unauthorized use of license-restricted code. The debate on the developers' rights to be informed about using their code for training language models still remains ongoing [4]. To address this issue, dataset curators often incorporate license restrictions when collecting code from open-source repositories. For example, The Stack [15], one of the largest publicly available code datasets, is comprised of over 6TB of source code spanning 358 programming languages and prioritizes permissively licensed code during data collection. However, enforcing license compliance remains a challenge, as malicious LLM developers can still extract and utilize license-restricted code for training, circumventing ethical and legal constraints. In this work, we consider a malicious scenario where an LLM developer intends to build a model for code by illegally using restricted code without being detected by Membership Inference (MI) techniques. We aim at assessing the risk of such a threat.

To mitigate such risks, recent studies have explored MI techniques as a detection mechanism [40, 42]. MI techniques aim to determine whether specific code snippets were included in a LLMs4Code's training data. Prior research has demonstrated that MI can effectively identify unauthorized usage of license-restricted code, highlighting its potential for auditing and enforcing compliance in large-scale model training [42].

Semantically Equivalent Code Transformation (SEPT) techniques pose a potential risk for malicious purposes because of their ability to obfuscate original code to look different (while preserving the same functionality). The rationale behind this concern is threefold. First, training LLMs on two semantically equivalent code datasets is likely to yield similar model performance. This means that malicious LLM developers relying on transformed restricted code face little risk of a significant performance drop. Second, SEPT may influence LLM memorization, potentially impacting the effectiveness of MI attacks. Recent findings support this claim, showing that LLMs exhibit strong memorization of specific patterns and training data samples [13, 43]. Third, a fundamental limitation of standard MI techniques is their reliance on exact match detection [3]. In other words, an attacker can potentially evade MI detection by applying SEPT—modifying code syntax while retaining its original functionality and naturalness. These transformations can obfuscate the presence of license-restricted code in the training data, complicating the detection of unauthorized data usage. Despite the potential impact of SEPT on MI, its effect has yet to be empirically validated. Hence, we propose our central research question (RQ):

How do semantically equivalent program transformation techniques impact membership inference?

To address this RQ, we first collected 23 SEPT rules applicable to Java to generate 23 new semantically equivalent Java datasets. We then fine-tuned two LLMs4Code, namely CodeGPT [20] and CodeGEN [25] on these 24 datasets (including the original one). The results demonstrate that, in the worst case, models fine-tuned on semantically equivalent datasets experienced only a 1.5% accuracy drop compared to those trained on the original dataset, confirming the feasibility of using such datasets as substitutes for fine-tuning.

Next, we evaluated the impact of SEPT on MI performance and found that one of the rules (namely, RenameVariable) led to an 8.01% decrease, raising concerns about SEPT's potential to bypass license enforcement. We then examined whether these rules exhibit cumulative effects on MI performance and found that they do not, which is reassuring. Finally, we conducted a causal analysis using a structural causal model to validate our findings, further strengthening the credibility of our conclusions. To summarize, the key contributions of this paper are as follows:

- **Impact of SEPT Rules on MI Performance:** We are the first to investigate the risks of using SEPT to bypass MI detection, which

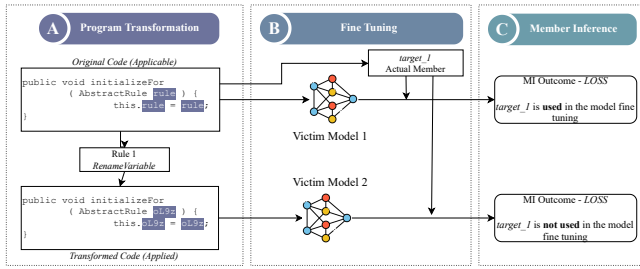


Figure 1: An example where a membership inference method successfully predicted membership before transformation but failed after transformation.

allows training code models without complying with licensing requirements. Our results demonstrate that *RenameVariable* rule has the most significant impact on MI performance in CodeGPT, in which 8.01% of the code remains undetected by MI, indicating a high risk of licensing breaches.

- **Causal Insights into Membership Inference Mitigation:** Our causal analysis confirms that *RenameVariable* rule has the strongest effect in reducing MI success, directly disrupting model memorization. While other transformations (e.g., control flow modifications) have moderate effects, combining multiple transformations does not provide additional reduction in MI effectiveness. Refutation tests validate these causal findings, supporting the reliability of the estimated causal effects.

- **Replication Package:** To support further research for evaluating MI risks and developing effective defenses, we publicly release our transformation rules, causal analysis tools, source code, and data used in the experiment¹.

2 Background

In this section, we outline the threat model that defines the adversary’s capabilities and objectives. We then review existing MI techniques, discuss the memorization phenomenon in LLMs4Codes for code generation, and introduce causal interpretability as a framework to analyze the effectiveness of SEPT in mitigating MI.

2.1 Threat Model

In most cases, LLMs4Codes, including those used for code completion, are deployed as black-box systems and provide services exclusively through API calls. In this study, we consider an adversary whose goal is to determine whether a given piece of code was included in the fine-tuning dataset of the target language model for code completion. The adversary has access only to the model’s generated outputs and logits (or loss) [3, 6, 22].

This threat model aligns with practical scenarios observed in commercial LLMs4Code providers, such as OpenAI’s GPT-4 [26], as well as in open-source LLMs4Codes, like StarCoder [18] and CodeGPT [20].

We provide an actual example in Fig. 1. The original code is used to fine-tune CodeGPT (victim model 1) and is accurately detected by *LOSS* (MI method, see Section 3) as a member, meaning it is correctly identified as being used in the fine-tuning process. However, after

applying *RenameVariable* rule to the original code, the transformed version is used to fine-tune another CodeGPT (victim model 2). Despite victim model 2 achieving similar performance to victim model 1, the *LOSS* mistakenly classifies the original code as a non-member. As a result, the malicious developer bypasses the licensing restrictions on the original code.

2.2 Membership Inference Techniques

MI is the task of determining whether a specific data sample was part of the training dataset for a machine learning model. Given a trained model, an adversary aims to exploit differences in its responses when presented with training (member) and non-training (non-member) samples. These attacks have raised concerns about the privacy of sensitive datasets in AI applications, including medical records, financial transactions, and code generation. MI is particularly concerning for LLMs trained on proprietary or sensitive codebases, where successful membership inference can reveal confidential or security-critical information [2].

MI can be categorized into white- and black-box approaches based on the level of knowledge available about the model and data. In a white-box MI, attackers have access to the model’s architecture, parameters, and the distribution of training data, allowing them to analyze internal states and gradients to infer membership. In contrast, a black-box MI operates without access to the model’s internals or training data distribution. Instead, attackers construct an attack model that learns to replicate the target model’s behavior based solely on its input-output interactions. In real-world scenarios, the black-box approach is more prevalent, as vendors typically do not expose their LLMs’ internal details to external users.

Current MI approaches can be broadly classified into two categories. The first category, direct analysis, infers membership by examining statistical patterns in the model’s predicted outputs. Initially introduced by Shokri et al. [36], these methods exploit differences in model behavior between training and unseen samples, often relying on confidence scores, loss values, or other statistical measures. Notable examples include *LOSS*, *ZLIB* compression, and *MIN_K*. The second category, classifier-based inference, extends this idea by using the model’s predicted output as input to a binary classifier trained to distinguish between member and non-member data, ultimately assigning a membership label (1 for member, 0 for non-member). Notable classifier-based methods include GOTCHA [42] and CODEMI [40].

2.3 Causal Interpretability

Causal interpretability, first introduced in do_{code} [23], is a post-hoc interpretability framework leveraging Pearl’s ladder of causation [28] to establish cause-and-effect relationships in model behavior for SE tasks, distinguishing genuine causal effects from spurious correlations. In the context of security for LLMs, we contextualize do_{code} to analyze how SEPT influences a model’s ability to retain or obscure training data. Specifically, we apply statistical and causal inference techniques to measure the *causal effect* of SEPT on MI outcomes. By formulating causal hypotheses using Structural Causal Model (SCM), we estimate the impact of transformations while controlling for confounders, ensuring a rigorous evaluation of whether

¹<https://anonymous.4open.science/r/final-38B2>

observed effects reflect genuine changes in MI effectiveness or superficial variations. Following do_{code} , we ① **model** the inference problem using causal assumptions encoded in graph representations, ② **identify** the causal estimand, ③ **estimate** causal effects via probabilistic and machine learning methods, and ④ **validate** causal estimates through sensitivity analyses, thereby ensuring the robustness of computed causal effects.

3 Methodology

In this section, we provide an overview of our methodology. Our experimental workflow is illustrated in Fig. 2. Our procedure begins with program transformation, in which we apply SEPT rules to modify the original code. Next, we fine-tune the victim models using either the original or transformed code. We then evaluate MI to assess the impact of these transformations on the success rate. Finally, we conduct a causal analysis to further examine our findings, offering insights into *how* and *why* certain transformations mitigate MI risks.

3.1 Semantically Equivalent Code Transformation

This study investigates whether code transformations pose a risk of enabling unauthorized use of license-restricted code. For this to hold, two conditions must be met: (1) the model’s performance after training should remain comparable to its original performance, and (2) the MI success rate should be reduced. If we can generate code that is functionally identical to the target code while appearing distinct, it should be possible to satisfy these conditions.

To achieve this, we synthesize semantically equivalent variants of the original code for training. Since determining program equivalence is undecidable [16], we employ hand-crafted transformation rules to ensure semantic preservation. In total, we collected 23 transformation rules, carefully designed and validated by previous work [17, 29, 45, 46]. These rules cover multiple levels of granularity, including naming, expression, and statement, ensuring a diverse set of transformations. A comprehensive list of these transformation rules is provided in Table 1. Finally, we apply these rules to the GitHub Java Corpus to generate semantically equivalent datasets.

3.2 Selected Membership Inference Techniques

In this study, the objective of MI is to determine whether a given code snippet x is part of the training dataset of an LLM, M . This involves computing a membership score $f(x, M)$, which is then thresholded to infer the membership status of x . We consider three MI techniques, each employing a distinct scoring function $f(x, M)$:

- **LOSS** ([44]): This attack assumes that training samples typically exhibit lower loss values compared to non-members. The membership score is computed using the model’s loss function:

$$f_{\text{loss}}(x, M) = \mathcal{L}(M, x) \quad (1)$$

where $\mathcal{L}$ represents the loss function (e.g., cross-entropy loss).

- **MIN_K probability** ([35]): This method ranks multiple model confidence scores and selects the k -th highest probability to determine membership. It is formulated as:

$$f_{\text{min-k}}(x, M) = \min_{i \leq k} P(y_i | x, M) \quad (2)$$

where $P(y_i | x, M)$ represents the model’s predicted probability for class y_i , and k is a tunable parameter.

- **ZLIB Compression** ([2]): It leverages Zlib compression for membership inference by exploiting sequence entropy. When compressing token sequences, **ZLIB** minimizes the number of bits used for representation. The **ZLIB** entropy of a sequence is defined as its compressed bit length. The attacker utilizes the ratio

$$\frac{\log(P_v)}{\text{zlib}(x)} \quad (3)$$

where P_v is the victim model’s perplexity, and Zlib entropy serves as a membership inference metric, capturing the relationship between model confidence and sequence compressibility.

3.3 Structural Causal Modeling

To investigate the causal relationship between code transformations in Table 1 and the effectiveness of MI, we follow the Causal Inference steps illustrated in Fig. 2. First, we propose a *SCM*. *SCMs* allows us to explicitly encode our causal hypothesis and analyze causal relationships among variables. By clearly defining treatments, outcomes, and confounders, *SCMs* facilitate precise estimation of causal effects while mitigating biases inherent in observational data. In particular, Fig. 3 illustrates the *SCM* proposed for our study, encoding our primary causal hypothesis: **altering the code used for fine-tuning through SEPT rules impacts the effectiveness of MI**.

In the proposed *SCM*, binary treatments (T), specifically the original code (T_0) and semantically equivalent transformed code (T_1), directly influence MI potential outcomes, measured by three metrics: Y_0 (**LOSS** in Eq. 1), Y_1 (**ZLIB** in Eq. 3) and Y_2 (**MIN_K** in Eq. 2). The causal effect between treatments and outcomes is influenced by confounders (Z), corresponding to variables extracted from the source code, such as `<nloc>` (i.e., number of lines of code), `<token_counts>` (i.e., number of tokens), `<#ast_levels>` (i.e., height of AST), `<#ast_nodes>`, `<#identifiers>`, `<#ast_errors>` (i.e., number of AST parsing errors), and `<code_complexity>`. In addition, variables present in the generated code serve as effect modifiers, influencing the magnitude or direction of the causal relationship between treatments and outcomes. Instrumental variables, commonly used to address confounding bias by isolating variations independent of confounders, were not considered in this study.

Computing ATEs: *Average Treatment Effect (ATE)* denoted by $(\lambda[\rho])$ in Fig. 3), quantifies the expected difference in MI outcomes between transformed and original code scenarios, averaged over the distribution of confounders and effect modifiers. Formally, the *ATE* is computed using Pearl’s *do-operator* [28] as $P(Y | do(T))$, representing an intervention scenario where treatments are set independently of confounders (e.g., Binary Treatment). This computation isolates the causal effect of code transformations by eliminating biases due to confounders. To estimate *ATEs*, a causal estimand is identified through graph-based criteria and the *adjustment formula*, determining the appropriate subset of confounders Z from the *SCM*. Specifically, the identification of the causal estimand leverages the *doWhy* library to systematically explore candidate causal estimands

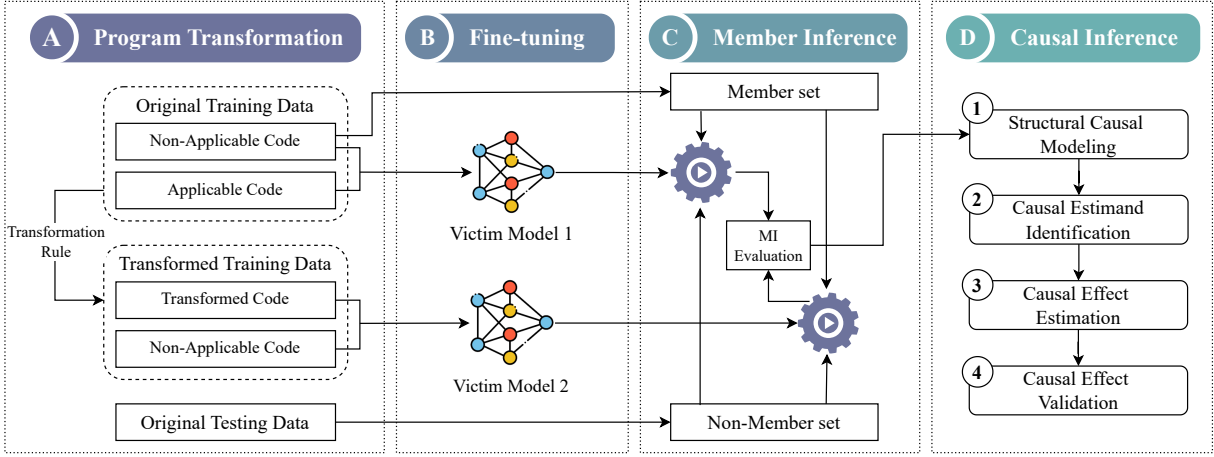


Figure 2: An overview of our study procedure. The diagram illustrates the workflow of our experiment, which begins with program transformation and fine-tuning, then proceeds to membership inference. Finally, we conduct a causal analysis to investigate the relationship between code transformations and the effectiveness of MI.

Table 1: Descriptions of code transformations at different levels.

ID	Transformation Rule	Description	Level
1	<i>RenameVariable</i>	Renames a variable while preserving functionality.	Naming
2	<i>For2While</i>	Converts a 'for' loop into a 'while' loop.	Statement
3	<i>While2For</i>	Converts a 'while' loop into a 'for' loop.	Statement
4	<i>Do2While</i>	Transforms a 'do-while' loop into a 'while' loop.	Statement
5	<i>IfElseIf2IfElse</i>	Refactors an 'if-else-if' chain into a nested 'if-else'.	Statement
6	<i>IfElse2IfElseIf</i>	Converts an 'if-else' structure into an 'if-else-if' chain.	Statement
7	<i>Switch2If</i>	Replaces a 'switch' statement with equivalent 'if-else' logic.	Statement
8	<i>Unary2Add</i>	Replaces a unary operation with an equivalent addition.	Expression
9	<i>Add2Equal</i>	Converts an addition assignment ('+=') into an equality assignment ('==').	Expression
10	<i>DivideVarDecl</i>	Splits multiple variable declarations into separate statements.	Expression
11	<i>MergeVarDecl</i>	Combines multiple variable declarations into a single statement.	Expression
12	<i>SwapStatement</i>	Swaps the order of two statements without data-flow dependencies.	Statement
13	<i>ModifyConstant</i>	Replaces a constant value with an equivalent mathematical expression.	Expression
14	<i>ReverseIf</i>	Reverses the condition in an 'if' statement and swaps its branches.	Statement
15	<i>If2CondExp</i>	Converts an 'if' statement into a conditional ('?:') expression.	Statement
16	<i>ConfExp2If</i>	Converts a conditional ('?:') expression into an 'if' statement.	Statement
17	<i>InfixDividing</i>	Splits an infix expression into smaller sub-expressions.	Expression
18	<i>DividePrePostFix</i>	Separates pre/postfix expressions into explicit statements.	Expression
19	<i>DividingComposedIf</i>	Breaks down a complex 'if' condition into multiple conditions.	Statement
20	<i>LoopIfContinue2Else</i>	Converts a 'continue' inside a loop into an 'else' branch.	Statement
21	<i>SwitchEqualExp</i>	Switch two sides of equality expression..	Expression
22	<i>SwitchStringEqual</i>	Switch two sides of String equality expression.	Expression
23	<i>SwitchRelation</i>	Switch two sides of relational expressions.	Expression

using criteria such as the back-door criterion, front-door criterion, instrumental variables, and mediation analysis [32].

Measuring ATE's Robustness: After computing the *ATE* of each transformation rule, we perform sensitivity analysis using multiple refutation techniques to validate the robustness of the obtained results. Specifically, we apply the following methods: (R_1) *introducing a random common cause* by adding an independent variable to check if the effect remains stable, ensuring it is not due to spurious correlations. (R_2) *applying placebo treatments* by replacing the actual treatment with a random one, where a near-zero effect confirms the validity of the original estimate. (R_3) *adding unobserved confounders* correlated with treatments and outcomes to assess

sensitivity, minimal effect change indicates robustness. And (R_4) performing *data subset* validation by checking effect consistency across subsets, where stability reinforces generalizability.

Interpreting ATEs: The *ATE* represents the expected difference in outcomes between treated and untreated groups, providing an estimate of causal impact. A positive *ATE* suggests that the applied transformation rule increases the effectiveness of MI, while a negative *ATE* indicates a reduction. The magnitude of the *ATE* reflects the strength of the effect, but its reliability depends on robustness checks and statistical significance.

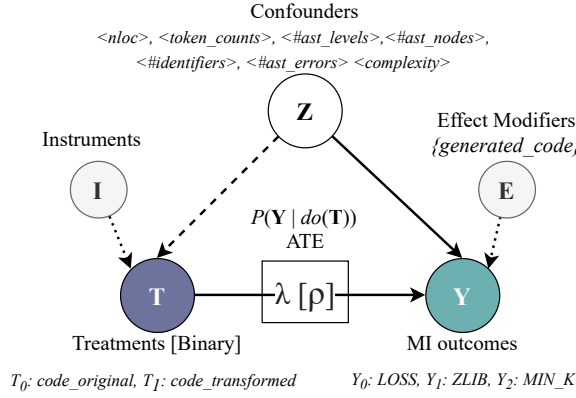


Figure 3: Structural Causal Model for MI.

4 Study Design

To systematically evaluate the impact of SEPT on MI against fine-tuned LLMs4Codes, we design a comprehensive empirical study. Our study investigates the effect of individual and combined transformations on MI effectiveness and provides a causal interpretation of their impact. This section outlines our research questions (RQs), the models and datasets used, evaluation metrics, causal analysis methodology, and implementation details.

4.1 Research Questions

RQ₁ [Impact of Individual Rules on MI] How does each transformation rule impact membership inference?

Motivation: When performing SEPT on a dataset, the transformed dataset should maintain the same code syntax and semantics as the original. However, since transformation rules alter the code differently, their effects vary. For instance, Rule 1 (*RenameVariable*) replaces variable names with randomly generated strings, while Rule 2 (*For2While*) alters the control flow by converting for loops into while loops. Moreover, each transformation rule introduces a distinct type of modification. Thus, it is essential to investigate the effect of various SEPT rules on MI performance.

RQ₂ [Impact of Combined Rules on MI] How do combined transformation rules impact membership inference?

Motivation: Applying more rules leads to a greater difference from the original code. Intuitively, this may make MI techniques less effective at distinguishing members from non-members. Moreover, considering that a transformation rule may not be applicable on a certain code, malicious LLM developers may wish to apply other rules. Hence, we aim to investigate if *cumulative effects* exist when multiple rules are applied.

RQ₃ [Interpret the Impact by Causality] To what extent does each transformation rule impact the success of MI?

Motivation: To validate the results of **RQ₁** and **RQ₂** and assess the true impact of each transformation on MI effectiveness, we move beyond correlation analysis to *causal inference*. Correlations may indicate relationships but cannot confirm whether transformations genuinely reduce MI success rate or if confounding factors influence the results.

4.2 Models

In this study, we focus on fine-tuning LLMs for code, as fine-tuning has been extensively studied in prior research [6, 40, 42]. For the downstream task, we select code completion, a core feature in nearly all modern integrated development environments. Code completion suggests relevant code based on the current coding context and is categorized into token-level, line-level, and block-level tasks. In line-level completion [8, 11], a model generates an entire line of code, whereas block-level completion produces larger structures such as functions or classes [5]. This study focuses on token-level completion, where the model predicts the next token given the preceding context. Our motivation stems from SEPT's primary effect on naming conventions, such as variable renaming [17]. Analyzing token-level completion allows us to examine how these transformations impact fine-tuned model's performance and the success rate of membership inference.

We do not use state-of-the-art models like OpenAI's Codex or ChatGPT, as their training data has not been publicly released, making it impossible to evaluate membership leakage [42]. Instead, we select CodeGPT [20] and CodeGEN [25] as our victim models. CodeGPT is a transformer-based language model with the same architecture and training objective as GPT-2 [30]. It is pre-trained on the CodeSearchNet dataset. Specifically, we use the *microsoft/CodeGPT-small-java* variant, trained on approximately 1.6M Java methods from CodeSearchNet. CodeGEN uses an autoregressive transformer architecture with next-token prediction as its learning objective. It is trained on a combination of natural language data and programming language datasets collected from GitHub. We select the *Salesforce/codegen-350M-multi* model, as it is pre-trained on BigQuery datasets and supports Java code completion.

4.3 Datasets

Since the victim models we selected are pre-trained on the CodeSearchNet and BigQuery datasets, we consider a widely used dataset from the CodeXGLUE benchmark: the GitHub Java Corpus, collected by Allamanis and Sutton [1]. This corpus consists of 14,807 open-source Java projects from GitHub. To maintain consistency, we follow the dataset preprocessing protocol established in prior studies [9, 12]. We extract a 1% subset of the corpus and split it into 12,934, 7,189, and 8,268 files for training, validation, and testing, respectively. This dataset is included in the CodeXGLUE benchmark [20], which is widely used to evaluate code completion models. CodeXGLUE provides both the original Java Corpus dataset and a preprocessing pipeline that tokenizes source code using a Java parser and removes comments. We apply transformation rules to the original Java Corpus training dataset and then follow the same preprocessing steps as CodeXGLUE to enable fair comparison.

We describe how we construct the member and non-member datasets for MI. Since our objective is to examine whether license-restricted code can be exploited in fine-tuning, the transformed code is only used for fine-tuning. When evaluating the MI success rate, we use the original dataset. Following the setup proposed by Duan et al. [3], we randomly select 1,000 samples from the GitHub Java Corpus training dataset that are applicable to a specific transformation rule as the member set. If fewer than 1,000 applicable

samples exist, we include all available samples. For example, only 307 data samples in the training set satisfy Rule 4 (Do2While), so we use all 307 samples as the member set. We then randomly select an equal number of samples from the GitHub Java Corpus testing dataset to form the non-member set. For each sample in both sets, we retain only code segments longer than 100 words, truncating them to a maximum of 200 words from the beginning, to create our benchmark examples [3].

4.4 Evaluation Metrics

Model performance: Accuracy is a standard metric for evaluating model performance in code completion [20, 40, 42]. We follow the evaluation strategy as CodeXGLUE. For a given sample, accuracy is computed as the ratio of correctly predicted tokens to the total number of tokens. Dataset-level accuracy is computed by averaging accuracy scores across all test samples. This metric reflects the performance of the victim model.

Membership inference performance: AUC ROC (Area under the ROC Curve) is a widely used metric for evaluating MI performance [3, 40]. MI is framed as a binary classification problem, where ground truth labels indicate whether a sample belongs to the training set (member) or the test set (non-member). The ROC curve illustrates the trade-off between the true positive rate and false positive rate at various classification thresholds. AUC ROC quantifies the model's ability to distinguish between members and non-members, with a higher AUC ROC indicating better MI performance. An AUC of 1 indicates perfect inference, while an AUC of 0.5 represents random guessing with no discriminative power.

4.5 Average Treatment Effect Computation

Given that we aim to understand the real effect of performing a SEPT rule on the success of MI, we calculate *ATEs* by formulating a causal intervention encompassing a binary treatment. Fig. 3 illustrates the *SCM* we devised to answer RQ3. More precisely, our control treatment T_0 (i.e., null hypothesis) consists of fine-tuning and computing the MI outcomes for a given architecture (i.e., CodeGEN, CodeGPT) using the original member dataset (refer to Sec. 4.3). Meanwhile, our treatment group T_1 (i.e., intervention) consists of fine-tuning and computing the MI outcomes for the same architecture (i.e., CodeGEN, CodeGPT) using the member dataset modified by the transformation.

The potential outcomes for MI under T_0 and T_1 come from different model fine-tuning settings for both CodeGEN and CodeGPT. Given that the distributions of values for each potential outcome in the T_0 and T_1 groups originate from different fine-tuning configurations, these values are not directly comparable. To address this issue, we compute relative scores for each potential outcome in both treatments.

To compute relative scores, we normalize each actual value by scaling it relative to the minimum and maximum values observed across all outcomes within the same treatment group. Specifically, for a given actual value α , we compute its relative score as:

$$\gamma = \frac{\alpha - \min(\alpha)}{\max(\alpha) - \min(\alpha) + \epsilon} \quad (4)$$

In Eq. 4, α represents the actual value, α denotes the set of all observed values within the treatment group, and ϵ is a small constant added to prevent division by zero. This transformation ensures that the relative scores range from 0 to 1, allowing for a standardized comparison across different fine-tuning settings. The relative score formula in Eq. 4 enables direct comparison of potential outcomes by normalizing them within a common scale.

4.6 Implementation Details

For fine-tuning CodeGPT, we use the publicly available code and hyperparameters from CodeXGLUE. Specifically, we set the learning rate to 8e-5 and save checkpoints every 500 steps, training for a total of 5 epochs. For CodeGEN, we follow the same fine-tuning process as CodeGPT, as it can be fine-tuned similarly to other causal LLMs like CodeGPT [25]. We set the learning rate to 1e-5, save checkpoints every 200 steps, and train for 2 epochs. We select the best-performing checkpoints based on their performance validation set. We adopt the implementation of different MI techniques from a popular Python library MIMIR [3], as it includes the most widely used MI methods and has been well developed. For a fair comparison, we reuse the default Mimir settings.

5 Empirical Results

We conduct experiments to address RQs regarding the impact of transformation rules on MI. For RQ1, we assess the influence of each transformation rule on model performance and the success rate of MI. For RQ2, we explore the effect of combining all transformation rules to increase their impact on MI. To answer RQ3, we examine the causal relationships between transformation rules and MI success.

5.1 RQ1: Impact of Individual Rule on MI

To answer RQ1, we construct a dataset for each transformation rule (described in Table 1) by applying the rule to the original dataset. For each rule, we apply it to all applicable code files in the original dataset. We provide a summary of the number of transformed data instances per rule in the replication package. We then fine-tune 24 models, each trained on either the original dataset or one of the 23 transformed datasets. We measure each rule's impact on task performance. We then apply three widely used MI techniques (described in Section 3.2) to assess whether an original code snippet was used for fine-tuning. To ensure fair evaluation, we use identical MI settings. Due to the page limit, we mainly present the top 5 most impactful rules to membership inference in Table 2 and attach the complete results in the replication package.

CodeGPT and CodeGEN, fine-tuned on the unmodified Java corpus, achieve test accuracies of 76.24% and 80.81%, respectively. By comparing models fine-tuned on the original and transformed datasets, we find that **all considered SEPT rules have minimal impact on the performance of both CodeGPT and CodeGEN**. More specifically, all rules result in task performance differences of no more than 1.5%. We report the AUC-ROC for the top five transformation rules for each MI method. Additionally, we compute the difference in MI performance between models fine-tuned on the transformed and original datasets, where a negative value ("-") indicates a decrease in MI performance.

Table 2: The 5 Most Impactful Transformation Rules to Membership Inference

Model	ID	Definition	Example		Test Accuracy	LOSS	AUC ROC(Δ)	
			Original	Transformed			MIN_K	ZLIB
CodeGPT	1	<i>RenameVariable</i>	Abstract plugin;	Abstract T1hnLQnIbr;	74.74	88.85 (-8.01)	92.67 (-5.30)	94.38 (-3.67)
	7	<i>Switch2If</i>	switch (var) {case a: Body A};	if (var == a) {Body A};	76.03	95.18 (-1.60)	96.88 (-1.10)	97.94 (-0.75)
	8	<i>Unary2Add</i>	var++;	var += 1;	76.14	94.16 (-1.17)	96.41 (-0.74)	97.79 (-0.46)
	13	<i>ModifyConstant</i>	var = 1;	var = 2 - 1;	76.19	92.15 (-1.07)	95.15 (-0.68)	96.35 (-0.51)
	4	<i>Do2While</i>	do {Body A}	{Body A};	76.17	94.59 (-1.44)	96.77 (-1.05)	97.52 (-0.75)
			while (condition a);	while (condition a) {Body A};				
CodeGEN	1	<i>RenameVariable</i>	Abstract plugin;	Abstract T1hnLQnIbr;	80.05	88.14 (-2.63)	90.4 (-2.13)	92.58 (-0.84)
	23	<i>SwitchRelation</i>	a > b;	b < a;	80.55	88.98 (-2.09)	91.45 (-1.51)	93.86 (-1.19)
	8	<i>Unary2Add</i>	var++;	var += 1;	80.88	85.54 (-1.14)	89.11 (-0.63)	93.21 (-0.53)
	21	<i>SwitchEqualExp</i>	a == b;	b == a;	80.62	91.36 (-1.03)	93.34 (-0.75)	95.39 (-0.52)
	17	<i>InfixDividing</i>	if (a==b && c==d) {Body A};	boolean vxtZ0 = a==b;	80.87	92.48 (-0.45)	94.5 (-0.22)	96.78 (-0.18)
				boolean ufZVZ = c==d;				
				if (vxtZ0 && ufZVZ) {Body A};				

Table 3: Combining All Transformation Rules and Their Impact

Model	Definition	Test Accuracy	LOSS	AUC ROC (Δ)	
				MIN_K	ZLIB
CodeGPT	Original	76.24	96.86	97.99	98.09
	Combined	74.65	93.46 (-3.40)	96.13 (-1.86)	96.94 (-1.15)
CodeGEN	Original	80.81	90.92	92.67	93.43
	Combined	79.65	88.73 (-2.19)	91.01 (-1.66)	93.12 (-0.31)

Table 4: Transformed Data Instances with Percentages by Rule Number (Top-5 are in bold)

ID	Transformed Instances (%)	ID	Transformed Instances (%)
1	12,143 (93.88%)	13	928 (7.17%)
2	3,006 (23.24%)	14	7,326 (56.64%)
3	1,650 (12.76%)	15	3,296 (25.48%)
4	286 (2.21%)	16	1,689 (13.06%)
5	2,859 (22.10%)	17	3,465 (26.79%)
6	682 (5.27%)	18	648 (5.01%)
7	1,376 (10.64%)	19	2,825 (21.84%)
8	1,459 (11.28%)	20	583 (4.51%)
9	909 (7.03%)	21	5,721 (44.23%)
10	269 (2.08%)	22	1,147 (8.87%)
11	2,425 (18.75%)	23	6,317 (48.84%)
12	6,990 (54.04%)		

The key observation is that Rule 1 (*RenameVariable*) has the most significant impact on MI performance in CodeGPT. *LOSS* decreases by 8.01%, indicating that 8.01% of data samples are assigned lower probability by the model than even unseen test samples. This is further supported by a 5.30% drop in *MIN_K*. This suggests that Rule 1 could be the most promising candidate for the malicious purpose. Fortunately, across all 23 transformation rules, the reduction in MI success rates—except for Rule 1—remains below 2.1%. Models fine-tuned on transformed datasets remain more familiar with pre-transformation data samples than with completely unseen samples, meaning that MI can still effectively detect fine-tuning set membership. The impact of Rule 1 on MI performance is smaller in CodeGEN than in CodeGPT, even though both models underwent the same dataset transformations and MI evaluations. This highlights that the pre-training dataset, the model architecture and the size of the model may significantly influence the effectiveness of the transformation rules in reducing the MI success rates.

RQ₁: Rule 1 (*RenameVariable*) is the only transformation that significantly reduces MI success rates when used in fine-tuning CodeGPT, whereas the other 22 rules provide little benefit in mitigating MI risks. Different **pretraining processes** and **model scales** may also affect a model’s susceptibility to MI.

5.2 RQ₂: Impact of Combined Rules on MI

To address RQ₂, we first design a method to determine the order in which to apply the rules. Some transformation rules can be applied together without conflict. For example, Rule 1 (*RenameVariable*) and Rule 2 (*For2While*) can be easily combined by simply first replacing variable names with random strings and then converting all for loops into while loops. However, certain transformation rules are mutually exclusive. For example, Rule 2 (*while-to-for*) and Rule 3 (*for-to-while*) conflict with each other. To maximize rule diversity in the final transformed dataset, we first count the number of data samples that can be transformed using each rule individually. Then, we apply transformation rules in ascending order of their applicable sample size. In other words, rules with fewer applicable samples are applied first. We report the number of data instance being transformed in the proposed order in Table 4. As a result, we obtain a transformed dataset which applied all the transformation rules, then we follow the same experimental procedure as in RQ₁ to evaluate performance.

Table 3 presents the performance of CodeGPT and CodeGEN fine-tuned on the dataset transformed using all 23 rules. The experimental results indicate that even when applying all transformation rules—leading to a dataset that significantly differs from the original one—the models can still achieve satisfactory performance. The task performance of CodeGPT and CodeGEN after fine-tuning on the fully transformed data is 74.65% and 79.65%, respectively. This represents only a slight decrease of 1.59% and 1.16% as compared to the original models.

Surprisingly, the impact on MI performance is not greater than that of individual transformation rules, particularly Rule 1. For CodeGPT, the MI performance degradation relative to the original model is 3.40%, 1.86%, and 1.15% on *LOSS*, *MIN_K*, and *ZLIB*, respectively—a smaller drop than that caused by Rule 1 alone. A similar trend is observed for CodeGEN, where MI performance decreases by 2.19%, 1.66%, and 0.31%, which is also less impactful

than applying Rule 1 alone. Our results show that the phenomenon of *cumulative effects* does not exist when applying multiple rules. Instead, applying all applicable rules even reduces the effectiveness of the most impactful rule.

RQ₂: Fine-tuning on the combined transformed dataset maintains high performance for both CodeGPT & CodeGEN. Yet, applying multiple SEPTs does not reduce MI success rates beyond the effect of the most impactful rule (Rule 1), indicating that the transformations **do not have a cumulative effect** on MI degradation.

5.3 RQ₃: Causal Effects of Rules on MI

To answer RQ₃, we compute the *ATEs* of each transformation rule on the effectiveness of MI for each potential outcome, as depicted in the proposed *SCM* (refer to Sec. 4.5). Results are summarized in Table 5 and Table 6 for CodeGEN and CodeGPT, respectively.

Our analysis reveals that among the 23 transformation rules, **Rule 1 (RenameVariable)** has the most significant impact in reducing MI effectiveness. Specifically, for CodeGEN, applying this transformation reduces the success of the *LOSS* outcome by an *ATE* of -0.173 and the *MIN_K* outcome by an *ATE* of -0.7836 . Similarly, in CodeGPT, *LOSS* decreases by -0.4619 and *MIN_K* by -0.9084 . These results indicate that renaming variables disrupts model memorization, making it harder for MI techniques to detect whether a code snippet was used in training.

While Rule 1 has the strongest effect, other transformations also contribute to reducing MI success rates. For example, **Rule 23 (SwitchRelation)** in CodeGEN lowers the effectiveness of *MIN_K* by -0.2614 , and **Rule 7 (Switch2If)** in CodeGPT reduces *MIN_K* success by -0.3694 . These results suggest that changes to control flow and expressions can introduce some level of uncertainty in model memorization, but their impact is weaker as compared to variable renaming.

A key observation from our study is that **combining multiple transformations does not lead to a cumulative reduction in MI effectiveness**. Instead, when all transformation rules are applied together, the MI success rate does not decrease significantly beyond what is achieved with Rule 1 alone. This suggests that while individual transformations can obscure MI detection, excessive transformations may introduce redundant modifications that do not further improve membership concealment.

To further validate these findings, we compare the *ATEs* with the Pearson correlation coefficients p reported in Table 5 and Table 6. The correlation values suggest that certain transformations, such as Rule 1, exhibit a strong negative correlation with MI success, which is supported by the *ATEs*. However, in some cases, the causal analysis contradicts the initial correlation assumptions. For instance, while Rule 22 (SwitchStringEqual) shows a weak correlation (≈ 0.1) with MI in Table 5, its *ATE* value indicates no measurable impact. This underscores the importance of causal inference in distinguishing genuine causal effects from spurious correlations.

Interestingly, a small number of transformations show negligible or even slightly positive causal effects on MI success. This suggests that certain structural modifications might unintentionally reinforce memorization instead of weakening it. This effect requires

further analysis, as it indicates that some transformations may increase the predictability of code patterns learned by the model. Future work should explore whether alternative transformation strategies could provide more reliable defenses against MI while maintaining model performance.

To ensure the robustness of our causal estimates, we apply multiple refutation methods (refer to Sec. 3.3), including R_1 (**random common cause**), R_2 (**placebo treatments**), R_3 (**unobserved confounders**), and R_4 (**data subset validation**). Refutation methods verify the reliability of our results. In Table 5 and Table 6, we report the results for R_1 . The results, close to zero for all *ATEs*, confirm that the estimated causal effects remain consistent, reinforcing the robustness of our conclusions.

RQ₃: Our causal analysis confirms that **Rule 1 (RenameVariable)** has the strongest impact in reducing MI success, while other transformations like **Rule 23 (SwitchRelation)** and **Rule 7 (Switch2If)** show moderate effects. **Combining multiple transformations does not further reduce MI effectiveness**. Refutation methods validate the robustness of these conclusions.

6 Discussion

In this section, we analyze our findings and explore their implications. We first highlight the potential of causal inference as a powerful tool for interpreting LLMs4Code. Next, we identify opportunities for improving MI. Additionally, we discuss broader challenges, including architectural vulnerabilities and the need for stronger licensing mechanisms. Finally, we outline potential threats to the validity of our results.

6.1 Implications and Future Work

#1 Causal inference can be an effective tool for interpreting LLM privacy for code. To the best of our knowledge, we are the first to propose causal inference as a framework for analyzing privacy in LLMs, specifically in the context of MI. Causal inference can help to interpret LLMs beyond spurious correlations. Our approach can also be extended to analyze other threats, such as *backdoor attacks*, by uncovering causal dependencies between malicious triggers and unintended model behaviors. Therefore, we advocate that the future work in the field of LLM for code can also leverage this technique to enhance the robustness and interpretability of LLMs4Code in security and privacy context.

#2 Customize MI with program transformation to detect unauthorized code usage in LLMs4Code training. As far as we are aware, none of the existing works has been specifically designed to protect code from being “stolen” by SEPT rules. Although several existing methods, such as adversarial regularization [24] and model ensemble [37], have been proposed to reduce the accuracy of MI, they do not account for the unique characteristics of code. In our experiments, we find that Rule 1 (RenameVariable) significantly reduces MI success rates, underscoring the distinct role of variable names in code. However, natural language carries different nature. For instance, changing a noun in a sentence would significantly alter the semantics (e.g., “Where is Canada?” vs. “Where is Japan?”). In programming languages, variable names can be modified without affecting functionality. Therefore, we argue that MI used to detect

Table 5: Causal Analysis (ATE) results for CodeGEN

ID	LOSS			ZLIB			MIN_K			ID	LOSS			ZLIB			MIN_K		
	<i>p</i>	ATE	<i>R</i> ₁	<i>p</i>	ATE	<i>R</i> ₁	<i>p</i>	ATE	<i>R</i> ₁		<i>p</i>	ATE	<i>R</i> ₁	<i>p</i>	ATE	<i>R</i> ₁	<i>p</i>	ATE	<i>R</i> ₁
1	-0.2315	-0.173	0.0019	-0.2	-0.0004	0	-0.2686	-0.7836	-0.0082	13	0.0941	0.0539	0.0029	0.0927	0.0001	0	0.0974	0.214	-0.0038
2	-0.0431	-0.0293	0.0028	-0.0485	-0.0001	0	-0.0442	-0.1212	-0.0054	14	-0.0466	-0.0245	-0.0003	-0.0467	-0.0001	0	-0.0547	-0.1123	-0.0037
3	-0.0031	-0.007	0.0016	-0.004	0	0	0.001	-0.0215	-0.0047	15	-0.0194	-0.0264	-0.0003	-0.0206	0	0	-0.0131	-0.09	0.0037
4	-0.0042	0.0129	0.0011	-0.0049	0.0001	0	0.0057	0.0622	-0.008	16	-0.02	0.0011	0.0005	-0.0225	0	0	-0.0182	0.0034	0
5	-0.0086	0.0013	-0.0011	-0.009	0	0	-0.0054	-0.0013	0.003	17	-0.0111	-0.0116	-0.0003	-0.0115	-0.0001	0	-0.0088	-0.0511	0.0068
6	0.086	0.0176	-0.0011	0.0897	0.0001	0	0.081	0.0673	0.0145	18	0.0898	0.0549	-0.002	0.0924	0.0001	0	0.0861	0.2	-0.0113
7	0.0108	-0.0018	-0.002	0.0112	0	0	0.0107	-0.007	0.0186	19	-0.0031	-0.0146	0	-0.0028	-0.0001	0	-0.0039	-0.0716	0.001
8	-0.1066	-0.0355	-0.0042	-0.1112	-0.0001	0	-0.1179	-0.1561	-0.006	20	0.0678	0.0105	-0.0001	0.0772	0	0	0.0725	0.0423	-0.015
9	0.0105	-0.0031	0.0016	0.0105	0	0	0.0121	-0.0107	-0.0046	21	-0.0201	-0.0095	-0.0002	-0.0184	0	0	-0.0201	-0.0383	0.0001
10	-0.0063	0.045	-0.001	-0.007	0.0001	0	-0.005	0.143	-0.0292	22	0.1001	0.0598	-0.0033	0.1009	0.0001	0	0.1012	0.2224	-0.0069
11	-0.0617	-0.0241	0.0014	-0.0666	-0.0001	0	-0.0636	-0.0946	0.0013	23	-0.1335	-0.0626	-0.0001	-0.1326	-0.0002	0	-0.1492	-0.2614	0.0035
12	-0.0065	-0.0044	-0.0014	-0.0072	0	0	0.0011	-0.0041	-0.0027	ALL	-0.2235	-0.1795	0.0007	-0.1886	-0.0004	0	-0.2397	-0.7463	0.0048

bold: – correlation, **bold underlined**: + correlation, **background blue**: – causal effect, **background green**: + causal effect

Table 6: Causal Analysis (ATE) results for CodeGPT

ID	LOSS			ZLIB			MIN_K			ID	LOSS			ZLIB			MIN_K		
	<i>p</i>	ATE	<i>R</i> ₁	<i>p</i>	ATE	<i>R</i> ₁	<i>p</i>	ATE	<i>R</i> ₁		<i>p</i>	ATE	<i>R</i> ₁	<i>p</i>	ATE	<i>R</i> ₁	<i>p</i>	ATE	<i>R</i> ₁
1	0.0419	0.0001	0	-0.4619	-0.0009	0	-0.5013	-0.9084	-0.0014	13	-0.0481	-0.0357	0.0006	-0.0485	-0.0001	0	-0.0432	-0.1102	0.007
2	-0.1098	-0.0448	0.0021	-0.121	-0.0001	0	-0.1112	-0.1637	-0.0045	14	-0.0374	0.0079	0.0012	-0.0383	0	0	-0.048	0.0106	-0.0046
3	0.0282	0.0321	0.0067	0.0297	0.0001	0	0.0262	0.1123	-0.0004	15	0.0386	0.025	-0.002	0.0379	0.0001	0	0.0412	0.0896	-0.0078
4	-0.1091	-0.0073	-0.0004	-0.1128	0	0	-0.1183	-0.0307	-0.0054	16	-0.0058	0.002	-0.0021	-0.0062	0	0	-0.0015	0.0139	-0.0018
5	0.002	0.011	0.0007	0.0008	0	0	0.0063	0.0535	-0.0055	17	-0.0504	-0.0232	-0.0023	-0.0522	-0.0001	0	-0.0491	-0.0831	0.0117
6	0.0012	0.0116	0.0052	0.0007	0	0	0.0028	0.0614	0.001	18	-0.0793	-0.0521	-0.0035	-0.0818	-0.0002	0	-0.079	-0.1978	-0.0003
7	-0.133	-0.1054	-0.0035	-0.1389	-0.0003	0	-0.1292	-0.3694	0.0119	19	0	0.0148	-0.0001	-0.0003	0	0	0.0001	0.0598	0.0006
8	-0.0973	-0.0533	0.0022	-0.1054	-0.0002	0	-0.0981	-0.1937	0.0074	20	0.0367	0.0389	0.0029	0.0419	0.0001	0	0.0418	0.1325	-0.0155
9	-0.0082	-0.0165	-0.0018	-0.0092	0	0	-0.0166	-0.0747	-0.0053	21	-0.022	-0.0143	0.0023	-0.0205	0	0	-0.0227	-0.0565	-0.0074
10	-0.0282	-0.1708	-0.0099	-0.0302	-0.0005	0	-0.0369	-0.7497	-0.0092	22	-0.0014	0.0238	0.0006	-0.002	0.0001	0	-0.0024	0.0903	-0.003
11	-0.0968	-0.0381	0.0003	-0.1023	-0.0001	0	-0.0989	-0.1553	-0.0066	23	-0.026	-0.0171	0.0022	-0.0268	0	0	-0.0259	-0.0617	-0.0016
12	-0.0708	-0.0506	0.0012	-0.0733	-0.0001	0	-0.0711	-0.1875	0.0071	ALL	-0.3058	-0.3131	-0.0011	-0.287	-0.0007	0	-0.319	-0.8525	-0.0005

bold: – correlation, **bold underlined**: + correlation, **background blue**: – causal effect, **background green**: + causal effect

membership in LLMs4Code context should handle variable names more carefully and strategically. In future work, we will design more advanced MI methods by leveraging SEPT. For example, we can use SEPT to expand a member set. Also, we will investigate the impact of assigning lower weights to variable names in MI methods and whether this can increase MI success rate.

#3 Tackle a weakness of the existing LLMs4Code architecture. Existing LLMs4Code commonly inherit the general Transformer architecture while pre-trained on code (domain-specific) data, such as CodeLlama [31], CodeGen [25]. Our results show that the architecture could be vulnerable for code-related tasks due to their sensitivity to SEPT transformations. More specifically, potential reasons can be due to the used tokenization algorithm and pretraining task. Shi et al. have demonstrated that existing popular tokenization algorithms carry certain limitation on code data [34]. More specifically, they found that Byte Pair Encoding algorithm cannot split the identifier names in a way that preserves the meaningful semantics. Another evidence is that, Zhou et al. proposed a backdoor attack for LLMs4Code by exploiting this limitation [41]. Their attack method mainly focuses on injecting the code backdoors by renaming the variable. Their results have shown that the designed code backdoors are hard to be detected by the defense methods. Hence, we advocate more future work to explore a new architecture which could be more resilient to SEPT. *Neurosymbolic AI* could address these limitations by integrating symbolic reasoning with deep learning models, allowing a deeper understanding of code semantics beyond token-level patterns. As an example, Velasco

et al. [38] proposed a novel Neurosymbolic framework for program comprehension that promises to improve model predictions for vulnerability classification tasks. Future work should explore extending such frameworks to enhance robustness against SEPT transformations in code-related tasks.

#4 Enhance licensing mechanisms for stronger compliance. The legal implications of using copyrighted data has been a focus of recent research. Previously, Katzy et al. [14] conducted an analysis of the SHA-256 hashes of such a dataset, which was constructed from 10,000 code repositories under these licenses. Their findings revealed substantial overlap between this strongly copy-left dataset and various widely used datasets, spanning 30 sources and 106 foundation models. However, our study demonstrates that the application of SEPT for obtaining training corpora would render such detection methods unreliable. This suggests that current provenance-based licensing tools may fail to detect code that has been modified through SEPT. Therefore, future research should focus on developing more resilient provenance detection techniques capable of accounting for SEPT scenarios to ensure proper attribution and compliance with licensing requirements.

6.2 Threats to Validity

Threats to **internal validity** refer to the correctness of our implementation. To mitigate the threats, we carefully implemented CodeGPT and CodeGEN by reusing the code released in the original repository² [20] and their models at HuggingFace. Our results with CodeGPT aligned with those reported in the original paper,

²<https://github.com/microsoft/CodeXGLUE>

further confirming the reliability of our implementations. We also reused the implementation of three MI methods from the author’s repository³ [3] and maintain their original MI settings. We also avoid threats to MI results from hyperparameter choices by adhering to the original MI implementation and reporting the average AUC-ROC based on 1,000 bootstrap samples for each attack.

Threats to **external validity** refer to the generalizability of our considered transformation rules. To mitigate this risk, we use a set of comprehensive rules, spanning various levels of transformations, including naming, statements, and expressions, and covering fundamental statement types, including if-else, while-loops, for-loops, variable declarations, assignments, and conditional expressions. Another threat may stem from model architecture, as the findings may not generalize beyond the architectures studied. Our study focuses on causal language models, specifically CodeGPT and CodeGEN. Given that GPT serves as the foundation for many state-of-the-art models and CodeGEN has been shown to perform well in code completion, these choices enhance the generalizability of our results. Our generalizability could also possibly affect by our target programming language (PL), *i.e.*, Java. We acknowledge that our findings may not be directly transferable to other PLs. However, we consider this threat to be minimal as our approach is language-independent. We leave the evaluation of additional PLs for future work.

Threats to **construct validity** refer to the suitability of our evaluation. We assess prediction accuracy on the same test dataset, aligning with the original paper’s setup. Additionally, we utilize AUC-ROC to evaluate the performance of MI, which is also widely adopted in many MI-related studies [3, 6, 42]. Therefore, we believe the threat is minimal.

7 Related Work

This section provides a summary of research related to this paper, covering (1) memorization in LLM for Code and (2) causal inference for software security and privacy.

7.1 Memorization in LLM for Code

In LLM for code context, memorization refers to the ability of LLMs to remember the training code samples. This phenomenon is particularly pronounced in models trained on datasets with high overlap, where identical or near-identical code samples appear frequently. When a model overfits on such data, it learns not only the patterns and structures of the code but also the exact instances, leading to direct regurgitation of training examples instead of synthesizing new, functionally equivalent code.

Memorization in LLMs raises significant privacy concerns, particularly regarding sensitive data exposure, licensing violations, and security vulnerabilities. For instance, Carlini et al. [2] demonstrated that LLMs can memorize and regurgitate sensitive user data, such as API keys and passwords, when trained on public repositories containing such information. This poses a severe privacy risk, as attackers can extract proprietary or confidential code snippets through targeted queries. Additionally, models trained on open-source code may inadvertently reproduce code under restrictive licenses (*e.g.*, GPL or proprietary licenses) without attribution,

³<https://github.com/iamgroot42/mimir>

leading to potential legal and ethical issues [14]. Security vulnerabilities are another major concern—if a model is trained on datasets containing insecure code patterns (*e.g.*, SQL injections, hardcoded credentials, or buffer overflows), it may learn and propagate these vulnerabilities in generated outputs, increasing the risk of privacy flaws in real-world applications [21].

Researchers employ various membership inference and extraction attacks to assess memorization in LLMs for code where an adversary attempts to determine if a given code snippet was part of the training set [2, 14]. However, none of the existing studies have considered how SEPT impacts memorization of LLMs4Code. In our context, particularly whether a code file, after modified with different variable names or code structure, still remain memorized.

7.2 Causal Inference for Software Security and Privacy

Recent research has increasingly leveraged causal inference to enhance cybersecurity in large language models (LLMs). Zhao et al. [47] introduced *Casper*, a causality-based analysis framework that systematically identifies vulnerabilities within LLM architectures by measuring causal influences of tokens, layers, and neurons on harmful outputs. Their approach uncovered latent vulnerabilities due to overfitting during reinforcement learning from human feedback (RLHF), highlighting how obscure input patterns could trigger unsafe behaviors. Similarly, Shen et al. [33] developed *BAIT*, a method utilizing causal interventions to detect Trojan backdoors within LLMs, effectively narrowing the search for malicious triggers by analyzing causal dependencies in autoregressive generation. Liu et al. [19] provided a broader perspective, outlining causal inference applications in diagnosing biases, ensuring robustness, and improving explainability of LLM outputs.

Collectively, the existing studies demonstrate that causal inference is a promising direction for understanding and securing the behavior of LLMs against adversarial threats. Beyond *do_{code}*, causal interpretability has influenced research on the evaluation and explanation of LLMs for code. SyntaxEval [39] applies causal interpretability to study syntactic learning in Masked Language Models, revealing negative causal effects between node types and model accuracy. ASTxplainer [27] extends this approach by aligning model predictions with AST structures, enhancing model evaluation and interpretability in large-scale code generation tasks.

8 Conclusions

In conclusion, this paper highlights a critical vulnerability in MI detection for LLMs4Code, demonstrating how SEPT can be exploited to evade MI attacks. We systematically analyze the impact of SEPT by applying 23 transformation rules for Java code and fine-tuning CodeGPT and CodeGEN on the transformed datasets. Our findings reveal that certain SEPT-based transformations reduce MI effectiveness, with Rule 1 (RenameVariable) lowering MI success by 8.01%, making it the most disruptive transformation. A causal analysis further confirms that variable renaming has the strongest impact on MI detection, while other transformations have only a moderate effect. Additionally, we find that combining multiple transformations does not further reduce MI effectiveness, suggesting that the transformation rules do not have a cumulative effect.

References

- [1] Miltiadis Allamanis and Charles Sutton. 2013. Mining source code repositories at massive scale using language modeling. In *2013 10th Working Conference on Mining Software Repositories (MSR)*. 207–216. doi:10.1109/MSR.2013.6624029
- [2] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*. 2633–2650.
- [3] Michael Duan, Anshuman Suri, Niloofar Miresghallah, Sewon Min, Weijia Shi, Luke Zettlemoyer, Yulia Tsvetkov, Yejin Choi, David Evans, and Hannaneh Hajishirzi. 2024. Do Membership Inference Attacks Work on Large Language Models?. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=av0D19pSkU>
- [4] duaneking. 2025. Project Uses Code Under Non-Permissive License to train AI models in violation of these licenses. <https://github.com/bigcode-project/starcoder/issues/9> Accessed: 6 May 2023.
- [5] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=hQwb-lbM6EL>
- [6] Wenjie Fu, Huangdong Wang, Chen Gao, Guanghua Liu, Yong Li, and Tao Jiang. 2024. Membership Inference Attacks against Fine-tuned Large Language Models via Self-prompt Calibration. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=PAWQvrForJ>
- [7] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The Pile: An 800GB Dataset of Diverse Text for Language Modeling. arXiv:2101.00027 [cs.CL] <https://arxiv.org/abs/2101.00027>
- [8] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smarandă Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 7212–7225. doi:10.18653/v1/2022.acl-long.499
- [9] Vincent J. Hellendoorn and Premkumar Devanbu. 2017. Are deep neural networks the best choice for modeling source code?. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (Paderborn, Germany) (ESEC/FSE 2017)*. Association for Computing Machinery, New York, NY, USA, 763–773. doi:10.1145/3106237.3106290
- [10] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2020. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. arXiv:1909.09436 [cs.LG] <https://arxiv.org/abs/1909.09436>
- [11] Maliheh Izadi, Roberta Gismonti, and Georgios Gousios. 2022. CodeFill: multi-token code completion by jointly learning from structure and naming sequences. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 401–412. doi:10.1145/3510003.3510172
- [12] Rafael-Michael Karampatsis, Hlib Babii, Romain Robbes, Charles Sutton, and Andrea Janes. 2020. Big code != big vocabulary: open-vocabulary models for source code. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (Seoul, South Korea) (ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 1073–1085. doi:10.1145/3377811.3380342
- [13] Anjan Karmakar, Julian Aron Prenner, Marco D'Ambros, and Romain Robbes. 2022. Codex Hacks HackerRank: Memorization Issues and a Framework for Code Synthesis Evaluation. arXiv:2212.02684 [cs.SE] <https://arxiv.org/abs/2212.02684>
- [14] Jonathan Katz, Razvan Popescu, Arie Van Deursen, and Maliheh Izadi. 2024. An Exploratory Investigation into Code License Infringements in Large Language Model Training Datasets. In *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering (Lisbon, Portugal) (FORGE '24)*. Association for Computing Machinery, New York, NY, USA, 74–85. doi:10.1145/3650105.3652298
- [15] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. 2022. The Stack: 3 TB of permissively licensed source code. *Preprint* (2022).
- [16] Dexter C. Kozen. 1977. *Rice's Theorem*. Springer Berlin Heidelberg, Berlin, Heidelberg, 245–248. doi:10.1007/978-3-642-85706-5_42
- [17] Thanh Le-Cong, Thanh-Dat Nguyen, Bach Le, and Toby Murray. 2025. Towards Reliable Evaluation of Neural Program Repair with Natural Robustness Testing. *ACM Trans. Softw. Eng. Methodol.* (Feb. 2025). doi:10.1145/3716167 Just Accepted.
- [18] Raymond Li, Loubna Ben allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltovozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Joel Lamy-Poirier, Joao Monteiro, Nicolas Gontier, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Ben Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason T Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Urvashi Bhattacharyya, Wenhao Yu, Sasha
- Luccioni, Paulo Villegas, Fedor Zhdanov, Tony Lee, Nadav Timor, Jennifer Ding, Claire S Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro Von Werra, and Harm de Vries. 2023. StarCoder: may the source be with you! *Transactions on Machine Learning Research* (2023). <https://openreview.net/forum?id=KoFOg41haE> Reproducibility Certification.
- [19] Xiaoyu Liu, Paiheng Xu, Junda Wu, Jiaxin Yuan, Yifan Yang, Yuhang Zhou, Fuxiao Liu, Tianrui Guan, Haoliang Wang, Tong Yu, Julian McAuley, Wei Ai, and Furong Huang. 2025. Large Language Models and Causal Inference in Collaboration: A Comprehensive Survey. doi:10.48550/arXiv.2403.09606 arXiv:2403.09606 [cs].
- [20] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, MING GONG, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie LIU. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *The fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*. <https://openreview.net/forum?id=6LE4dQXaUcb>
- [21] Vahid Majdinasab, Michael Joshua Bishop, Shawn Rasheed, Arghavan Moradikhah, Amjed Tahir, and Foutse Khomh. 2024. Assessing the Security of GitHub Copilot's Generated Code - A Targeted Replication Study . In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE Computer Society, Los Alamitos, CA, USA, 435–444. doi:10.1109/SANER60148.2024.00051
- [22] Justus Mattern, Fatemehsadat Miresghallah, Zhijing Jin, Bernhard Schoelkopf, Mrinmaya Sachan, and Taylor Berg-Kirkpatrick. 2023. Membership Inference Attacks against Language Models via Neighbourhood Comparison. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 11330–11343. doi:10.18653/v1/2023.findings-acl.719
- [23] David Nader Palacio, Alejandro Velasco, Nathan Cooper, Alvaro Rodriguez, Kevin Moran, and Denys Poshyvanyk. 2024. Toward a Theory of Causation for Interpreting Neural Code Models . *IEEE Transactions on Software Engineering* 50, 05 (May 2024), 1215–1243. doi:10.1109/TSE.2024.3379943
- [24] Milad Nasr, Reza Shokri, and Amir Houmansadr. 2018. Machine Learning with Membership Privacy using Adversarial Regularization. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (Toronto, Canada) (CCS '18)*. Association for Computing Machinery, New York, NY, USA, 634–646. doi:10.1145/3243734.3243855
- [25] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=iaYcKpY2B>
- [26] OpenAI and Josh Achiam et al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] <https://arxiv.org/abs/2303.08774>
- [27] David N. Palacio, Alejandro Velasco, Daniel Rodriguez-Cardenas, Kevin Moran, and Denys Poshyvanyk. 2023. Evaluating and Explaining Large Language Models for Code Using Syntactic Structures. doi:10.48550/arXiv.2308.03873 arXiv:2308.03873 [cs].
- [28] Judea Pearl and Dana Mackenzie. 2018. *The Book of Why: The New Science of Cause and Effect* (1st ed.). Basic Books, Inc., USA.
- [29] Md Rafiqul Islam Rabin, Nghi DQ Bui, Ke Wang, Yijun Yu, Lingxiao Jiang, and Mohammad Amin Alipour. 2021. On the generalizability of neural program models with respect to semantic-preserving program transformations. *Information and Software Technology* 135 (2021), 106552.
- [30] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [31] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] <https://arxiv.org/abs/2308.12950>
- [32] Amit Sharma and Emre Kiciman. 2020. DoWhy: An End-to-End Library for Causal Inference. *arXiv preprint arXiv:2011.04216* (2020).
- [33] Guangyu Shen, Siyuan Cheng, Zhuo Zhang, Guan hong Tao, Kaiyuan Zhang, Hanxi Guo, Lu Yan, Xiaolong Jin, Shengwei An, Shiqing Ma, and Xiangyu Zhang. 2025. BAIT: Large Language Model Backdoor Scanning by Inverting Attack Target . In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 102–102. doi:10.1109/SP61157.2025.00103
- [34] Jieke Shi, Zhou Yang, Junda He, Bowen Xu, and David Lo. 2022. Can identifier splitting improve open-vocabulary language model of code?. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 1134–1138.

- [35] Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. 2024. Detecting Pretraining Data from Large Language Models. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=zWqr3MQuNs>
- [36] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership Inference Attacks Against Machine Learning Models. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–18. doi:10.1109/SP.2017.41
- [37] Xinyu Tang, Saeed Mahloujifar, Liwei Song, Virat Shejwalkar, Milad Nasr, Amir Houmansadr, and Prateek Mittal. 2022. Mitigating Membership Inference Attacks by Self-Distillation Through a Novel Ensemble Architecture. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 1433–1450. <https://www.usenix.org/conference/usenixsecurity22/presentation/tang>
- [38] Alejandro Velasco, Aya Garrryyeva, David N. Palacio, Antonio Mastropaolo, and Denys Poshyvanyk. 2025. Toward Neurosymbolic Program Comprehension. doi:10.48550/arXiv.2502.01806 arXiv:2502.01806 [cs].
- [39] Alejandro Velasco, David N. Palacio, Daniel Rodriguez-Cardenas, and Denys Poshyvanyk. 2024. Which Syntactic Capabilities Are Statistically Learned by Masked Language Models for Code?. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results (Lisbon, Portugal) (ICSE-NIER'24)*. Association for Computing Machinery, New York, NY, USA, 72–76. doi:10.1145/3639476.3639768
- [40] Yao Wan, Guanghua Wan, Shijie Zhang, Hongyu Zhang, Pan Zhou, Hai Jin, and Lichao Sun. 2024. Does Your Neural Code Completion Model Use My Code? A Membership Inference Approach. arXiv:2404.14296 [cs.SE] <https://arxiv.org/abs/2404.14296>
- [41] Zhou Yang, Bowen Xu, Jie M Zhang, Hong Jin Kang, Jieke Shi, Junda He, and David Lo. 2024. Stealthy backdoor attack for code models. *IEEE Transactions on Software Engineering* 50, 4 (2024), 721–741.
- [42] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, DongGyun Han, and David Lo. 2024. Gotcha! This Model Uses My Code! Evaluating Membership Leakage Risks in Code Models. *IEEE Transactions on Software Engineering* 50, 12 (Dec. 2024), 3290–3306. doi:10.1109/TSE.2024.3482719
- [43] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, Donggyun Han, and David Lo. 2024. Unveiling Memorization in Code Models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 72, 13 pages. doi:10.1145/3597503.3639074
- [44] Samuel Yeom, Irene Giacomelli, Matt Fredrikson, and Somesh Jha. 2018. Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. IEEE Computer Society, Los Alamitos, CA, USA, 268–282. doi:10.1109/CSF.2018.00027
- [45] Shiwen Yu, Ting Wang, and Ji Wang. 2022. Data augmentation by program transformation. *Journal of Systems and Software* 190 (2022), 111304.
- [46] Weiwei Zhang, Shengjian Guo, Hongyu Zhang, Yulei Sui, Yinxing Xue, and Yun Xu. 2023. Challenging machine learning-based clone detectors via semantic-preserving code transformations. *IEEE Transactions on Software Engineering* 49, 5 (2023), 3052–3070.
- [47] Wei Zhao, Zhe Li, and Jun Sun. 2023. Causality Analysis for Evaluating the Security of Large Language Models. doi:10.48550/arXiv.2312.07876 arXiv:2312.07876 [cs].